Introduction to



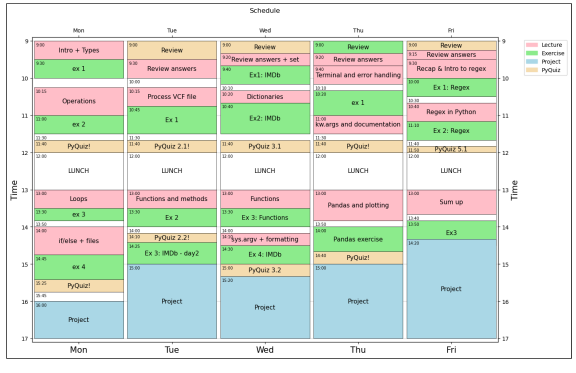
with Application to Bioinformatics

- Day 1

Practical issues

- Course website: https://uppsala.instructure.com/courses/126155
- Course lectures streamed from Uppsala, Umeå, and Lund
- TAs on each site
- Short lectures with many breaks
- Schedule times are approximate

Schedule


Schedule
Mon
Tue
Wed
Thu
Fri
Time
Lecture
Exercise
Project
PyQuiz
Intro + Types
ex 1
Operations
ex 2
PyQuiz!
LUNCH
Loops
ex 3
if/else + files
ex 4
PyQuiz!
Project
Review
Review answers
Process VCF file
Ex 1
PyQuiz 2.1!
LUNCH
Functions and methods
Ex 2
PyQuiz 2.2!
Ex 3: IMDb - day2
Project
Review
Review answers + set
Ex1: IMDb
Dictionaries
Ex2: IMDb
PyQuiz 3.1
LUNCH
Functions
Ex 3: Functions
sys.argv + formatting
Ex 4: IMDb
PyQuiz 3.2
Project
Review
Review answers
Terminal and error handling
ex 1
kwargs and documentation
PyQuiz!
LUNCH
Pandas and plotting
Pandas exercise
PyQuiz!
Project
Review
Review answers
Recap & Intro to regex
Ex 1: Regex
Regex in Python
Ex 2: Regex
PyQuiz 5.1
LUNCH
Sum up
Ex3
Project

To start with

- Has everyone managed to log in to Canvas?
- Has everyone managed to install Python?
- Have you managed to run the test script?
- Have you installed notebooks? (optional)
- Canvas tour
- PyQuizzes

What is programming?

Wikipedia:

"Computer programming is the process of building and designing an executable computer program for accomplishing a specific computing task"

What can we use it for?

Endless possibilities!

- reverse complement DNA
- custom filtering of VCF files
- plotting of results
- all excel stuff!

Why Python?

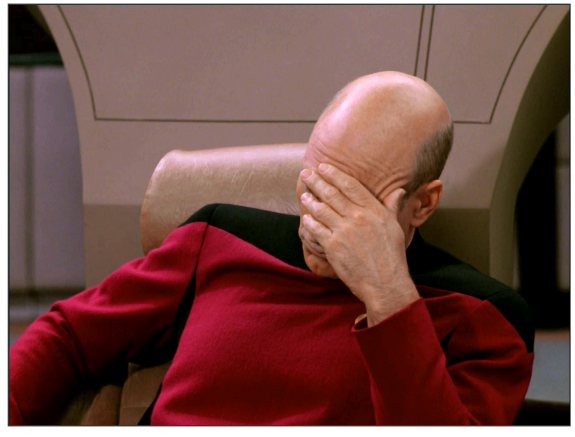
Typical workflow

1. Get data
2. Clean, transform data in spreadsheet
3. Copy-paste, copy-paste, copy-paste
4. Run analysis & export results
5. Realise the columns were not sorted correctly
6. Go back to step 2, Repeat

Why Python?

Typical workflow

1. Get data
2. Clean, transform data in spreadsheet
3. Copy-paste, copy-paste, copy-paste
4. Run analysis & export results
5. Realise the columns were not sorted correctly
6. Go back to step 2, Repeat



Python versions

Old versions	Python 3
Python 1.0 - January 1994	Python 3.0 - December 3, 2008
Python 1.0 - January 1994	Python 3.1 - June 27, 2009
Python 1.2 - April 10, 1995	Python 3.2 - February 20, 2011
Python 1.3 - October 12, 1995	Python 3.3 - September 29, 2012
Python 1.4 - October 25, 1996	Python 3.4 - March 16, 2014
Python 1.5 - December 31, 1997	Python 3.5 - September 13, 2015
Python 1.6 - September 5, 2000	Python 3.6 - December 23, 2016
Python 2.0 - October 16, 2000	Python 3.7 - June 27, 2018
Python 2.1 - April 17, 2001	Python 3.8 - October 14, 2019
Python 2.2 - December 21, 2001	Python 3.9 - October 5, 2020
Python 2.3 - July 29, 2003	Python 3.10 - October 4, 2021
Python 2.4 - November 30, 2004	Python 3.11 - October 24 2022
Python 2.5 - September 19, 2006	Python 3.12 - October 2 2023
Python 2.6 - October 1, 2008	Python 3.13 - October 2024
Python 2.7 - July 3, 2010	Python 3.14 - October 2025
	Python 3.15 - October 2026

Course content

- Core concepts about Python syntax: Data types, blocks and indentation, variable scoping, iteration, functions, methods and arguments
- Different ways to control program flow using loops and conditional tests
- Regular expressions and pattern matching
- Writing functions and best-practice ways of making them usable
- Reading from and writing to files
- Code packaging and Python libraries
- How to work with biological data using external libraries.

Learning outcomes

At the end of the course, you should be able to:

- Use variables and exlain how operators work
- Process data using loops
- Separate data using if/else statements
- Use functions to read and write to files
- Describe their own approach to a coding task
- Understand the difference between functions and methods
- Be able to read the documentation for built-in functions/methods
- Give examples of use cases for dictionaries
- Write data to a simple dictionary
- Understand the concept and syntax of a function

Learning outcomes, cont.

At the end of the course, you should be able to:

- Write basic functions for processing data
- Describe pandas dataframes
- Give examples of how to use pandas for processing data
- Explain how regex can be used
- Define the python syntax for regex
- Combine basic concepts to create functional stand-alone programs to process data
- Write file processing Python programs that produce output to the terminal and/or external files
- Explain how to debug and further develop your skills in Python after the course

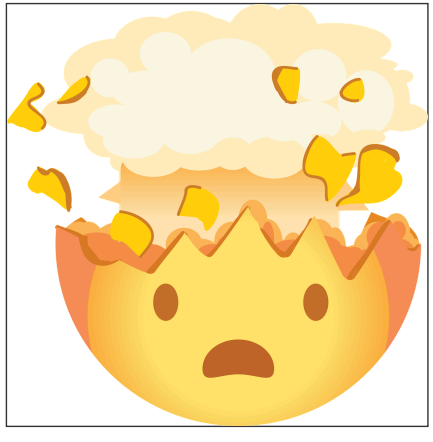
Some good advice

- 5 days to learn Python is not much
- Amount of information will decrease over days
- Complexity of tasks will increase over days
- Read the error messages!
- Save all your code

How to seek help:

- Google
- Ask AI
- Ask your neighbour
- Ask an assistant

You will look like this:



Day 1

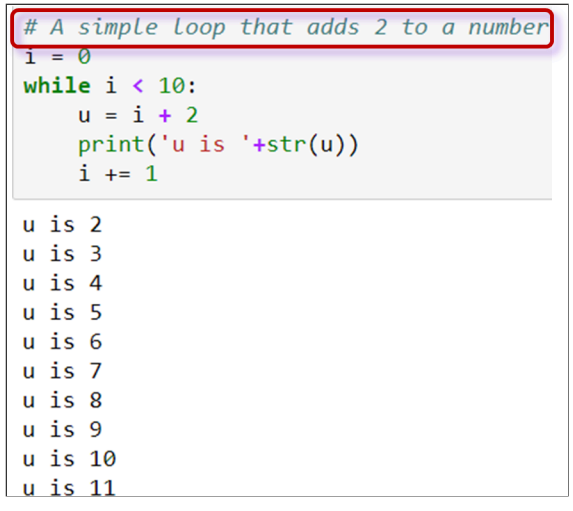
- Types and variables
- Operations
- Loops
- if/else statements

Example of a simple Python script

In [29]:
```
# A simple loop that adds 2 to a number
i = 0
while i < 10:
    u = i + 2
    print('u is' + str(u))
    i += 1
```

```
u is2
u is3
u is4
u is5
u is6
u is7
u is8
u is9
u is10
u is11
```

Example of a simple Python script



```
# A simple loop that adds 2 to a number
i = 0
while i < 10:
    u = i + 2
    print('u is '+str(u))
    i += 1
```

```
u is 2
u is 3
u is 4
u is 5
u is 6
u is 7
u is 8
u is 9
u is 10
u is 11
```

Comment

All lines starting with # is interpreted by python as a comment and are not executed. Comments are important for documenting code and considered good practise when doing all types of programming

Example of a simple Python script

```
# A simple loop that adds 2 to a number
i = 0
while i < 10:
    u = i + 2
    print('u is '+str(u))
    i += 1
```

```
u is 2
u is 3
u is 4
u is 5
u is 6
u is 7
u is 8
u is 9
u is 10
u is 11
```

Literals

All literals have a type:

- Strings (str) ‘Hello’ “Hi”
- Integers (int) 5
- Floats (float) 3.14
- Boolean (bool) True or False

Literals define values

In [30]:
```
'this is a string'
"this is also a string"
3       # here we can put a comment so we know that this is an integer
3.14    # this is a float
True    # this is a boolean

type(True)
```

Out[30]:
```
bool
```

Literals define values

In [30]:
```
'this is a string'
"this is also a string"
3       # here we can put a comment so we know that this is an integer
3.14    # this is a float
True    # this is a boolean

type(True)
```

Out[30]:
```
bool
```

Collections

In [31]:
```
[3, 5, 7, 4, 99]       # this is a list of integers

('a', 'b', 'c', 'd')   # this is a tuple of strings
{'a', 'b', 'c'}        # this is a set of strings
{'a':3, 'b':5, 'c':7}  # this is a dictionary with strings as keys and i

type([3, 5, 7, 4, 99])
```

Out[31]:
```
list
```

What operations can we do with different values?

That depends on their type:

In [32]:
```
'a string'+' another string'
2 + 3.4
'a string ' * 3
'a string ' * 3.4
```

```
---------------------------------------------------------------------
----------
TypeError                                 Traceback (most recent
call last)
Input In [32], in <cell line: 4>()
      2 2 + 3.4
      3 'a string ' * 3
----> 4 'a string ' * 3.4

TypeError: can't multiply sequence by non-int of type 'float'
```

What operations can we do with different values?

That depends on their type:

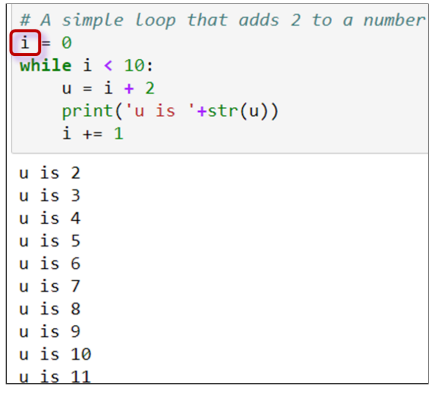
In [32]:
```
'a string'+' another string'
2 + 3.4
'a string ' * 3
'a string ' * 3.4
```

```
---------------------------------------------------------------------
----------
TypeError                                 Traceback (most recent
call last)
Input In [32], in <cell line: 4>()
      2 2 + 3.4
      3 'a string ' * 3
----> 4 'a string ' * 3.4

TypeError: can't multiply sequence by non-int of type 'float'
```

Type	Operations
int	+ - * / ** % // ...
float	+ - * / ** % // ...
string	+ *

Example of a simple Python script



```
# A simple loop that adds 2 to a number
i = 0
while i < 10:
    u = i + 2
    print('u is '+str(u))
    i += 1
```

```
u is 2
u is 3
u is 4
u is 5
u is 6
u is 7
u is 8
u is 9
u is 10
u is 11
```

Identifiers

Identifiers are used to identify a program element in the code.

For example:

- Variables
- Functions
- Modules
- Classes

Variables

Used to store values and to assign them a name.

Examples:

- `i       = 0`
- `counter = 5`
- `snpname = 'rs2315487'`
- `snplist = ['rs21354', 'rs214569']`

Variables

Used to store values and to assign them a name.

Examples:

- `i       = 0`
- `counter = 5`
- `snpname = 'rs2315487'`
- `snplist = ['rs21354', 'rs214569']`

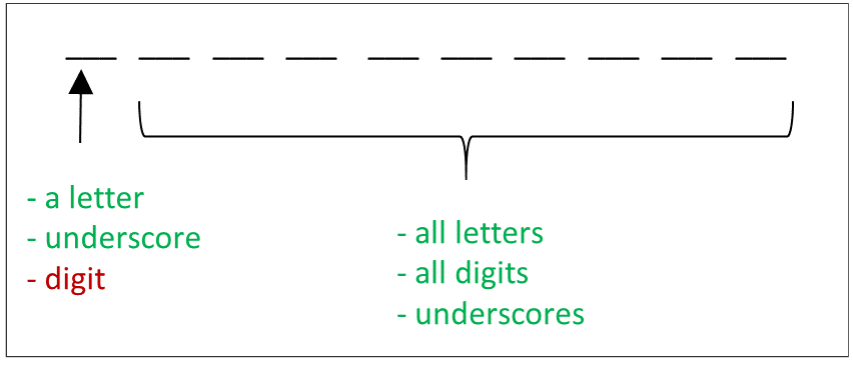
In [33]:
```
width  = 42
height = 20

snpname = 'rs56483 '
snplist = ['rs12345','rs458782']

snpname * 3
width * height
```

Out[33]:
```
840
```

How to correctly name a variable



Allowed:

Var_name
_total
aReallyLongName
with_digit_2
dkfsjdsklut *(well, allowed, but NOT recommended)*

Not allowed:

2save
*important
Special%
With spaces

NO special characters:

+ - * $ % ; : , ? ! { } ( ) < > “ ‘ | \ / @

Reserved keywords

False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

These words can not be used as variable names

Summary

- Comment your code!
- Literals define values and can have different types (strings, integers, floats, boolean)
- Values can be collected in lists, tuples, sets, and dictionaries
- The operation that can be performed on a certain value depends on the type
- Variables are identified by a name and are used to store a value or collections of values
- Name your variables using descriptive words without special characters and reserved keywords

→ **Notebook Day_1_Exercise_1 (~30 minutes)**

NOTE!

How to get help?

- Google and Stack overflow are your best friends!
- Official python documentation
- Ask your neighbour
- Ask us

Python standard library

Built-in Functions				
abs( )	delattr( )	hash( )	memoryview( )	set( )
all( )	dict( )	help( )	min( )	setattr( )
any( )	dir( )	hex( )	next( )	slice( )
ascii( )	divmod( )	id( )	object( )	sorted( )
bin( )	enumerate( )	input( )	oct( )	staticmethod( )
bool( )	eval( )	int( )	open( )	str( )
breakpoint( )	exec( )	isinstance( )	ord( )	sum( )
bytearray( )	filter( )	issubclass( )	pow( )	super( )
bytes( )	float( )	iter( )	print( )	tuple( )
callable( )	format( )	len( )	property( )	type( )
chr( )	frozenset( )	list( )	range( )	vars( )
classmethod( )	getattr( )	locals( )	repr( )	zip( )
compile( )	globals( )	map( )	reversed( )	__import__( )
complex( )	hasattr( )	max( )	round( )	

Example `print()` and `str()`

```
# A simple loop that adds 2 to a number
i = 0
while i < 10:
    u = i + 2
    print('u is '+str(u))
    i += 1
```

```
u is 2
u is 3
u is 4
u is 5
u is 6
u is 7
u is 8
u is 9
u is 10
u is 11
```

Note!
Here we format everything to a string before printing it

Python standard library

Built-in Functions				
abs( )	delattr( )	hash( )	memoryview( )	set( )
all( )	dict( )	help( )	min( )	setattr( )
any( )	dir( )	hex( )	next( )	slice( )
ascii( )	divmod( )	id( )	object( )	sorted( )
bin( )	enumerate( )	input( )	oct( )	staticmethod( )
bool( )	eval( )	int( )	open( )	str( )
breakpoint( )	exec( )	isinstance( )	ord( )	sum( )
bytearray( )	filter( )	issubclass( )	pow( )	super( )
bytes( )	float( )	iter( )	print( )	tuple( )
callable( )	format( )	len( )	property( )	type( )
chr( )	frozenset( )	list( )	range( )	vars( )
classmethod( )	getattr( )	locals( )	repr( )	zip( )
compile( )	globals( )	map( )	reversed( )	__import__( )
complex( )	hasattr( )	max( )	round( )	

In [34]:
```
width  = 5
height = 3.6
snps   = ['rs123', 'rs5487']
snp    = 'rs2546'
```

```
active = True
nums   = [2,4,6,8,4,5,2]

int(height)
```

Out[34]: 3

More on operations

Operation	Result
x + y	sum of x and y
x - y	difference between x and y
x ** y	x to the power y
....	....
pow(x, y)	x to the power y
float(x)	x converted to float
int(x)	x converted to int!
len(z)	length of z if list
max(z)	maximum in list of z
min(z)	minimum in list of z

In [35]:
```
x = 4
y = 3
```

```
z = [2, 3, 6, 3, 9, 23]
pow(x, y)
```

Out[35]: 64

Comparison operators

Operation	Meaning
<	less than
<=	less than or equal
>	greater than
>=	greater than or equal
==	equal
!=	not equal

Can be used on int, float, str, and bool. Outputs a boolean.

In [36]:
```
x = 5
y = 3

y != x
```

Out[36]:
```
True
```

Logical operators

Operation	Meaning
and	connects two statements, both conditions having to be fulfilled
or	connects two statements, either conditions having to be fulfilled
not	reverses and/or

Membership operators

Operation	Meaning
in	value in object
not in	value not in object

In [37]:
```
x = 2
y = 3
x == 2 and y == 5
```

```
x = [2,4,7,3,5,9]
y = ['a','b','c']

2 in x
4 in x and 'd' in y
```

Out[37]:

```
False
```

In [38]:

```
# A simple loop that adds 2 to a number and checks if the number is even
i    = 0
even = [2,4,6,8,10]
while i < 10:
    num = i + 2
    print('num is '+str(num)+'. Is this number even? '+str(num in even))
    i += 1
```

```
num is 2. Is this number even? True
num is 3. Is this number even? False
num is 4. Is this number even? True
num is 5. Is this number even? False
num is 6. Is this number even? True
num is 7. Is this number even? False
num is 8. Is this number even? True
num is 9. Is this number even? False
num is 10. Is this number even? True
num is 11. Is this number even? False
```

In [39]:

```
# A simple loop that adds 2 to a number, check if number is even and bel
i    = 0
even = [2,4,6,8,10]
while i < 10:
    num = i + 2
    print('num is '+str(num)+'. Is this number even and below 5? '+\
          str(num in even and num < 5))
    i += 1
```

```
num is 2. Is this number even and below 5? True
num is 3. Is this number even and below 5? False
num is 4. Is this number even and below 5? True
num is 5. Is this number even and below 5? False
num is 6. Is this number even and below 5? False
num is 7. Is this number even and below 5? False
num is 8. Is this number even and below 5? False
num is 9. Is this number even and below 5? False
num is 10. Is this number even and below 5? False
num is 11. Is this number even and below 5? False
```

Order of precedence

There is an order of precedence for all operators:

Operators	Descriptions
`**`	exponent
`*`, `/`, `%`	multiplication, division, modulo
`+`, `-`	addition, substraction
`<`, `<=`, `>=`, `>`	comparison operators
`==`, `!=`, in, not in	comparison operators
not	boolean NOT
and	boolean AND
or	boolean OR

Word of caution when using operators

Word of caution when using operators

In [40]:
```
x = 5
y = 7
z = 2
x == 5 and y < 7 or z > 1

# and binds stronger than or
x > 4 or y == 6 and z > 3
x > 4 or (y == 6 and z > 3)
(x > 4 or y == 6) and z > 3
```

Out[40]:
```
False
```

Word of caution when using operators

In [40]:
```
x = 5
y = 7
z = 2
x == 5 and y < 7 or z > 1

# and binds stronger than or
x > 4 or y == 6 and z > 3
x > 4 or (y == 6 and z > 3)
(x > 4 or y == 6) and z > 3
```

Out[40]:
```
False
```

In [41]:
```
# BEWARE!
x = 5
y = 8

#xx == 6 or xxx == 6 or x > 2
x > 42 and (xx > 1000 or y < 7)
```

Out[41]:
```
False
```

Word of caution when using operators

In [40]:
```
x = 5
y = 7
z = 2
x == 5 and y < 7 or z > 1

# and binds stronger than or
x > 4 or y == 6 and z > 3
x > 4 or (y == 6 and z > 3)
(x > 4 or y == 6) and z > 3
```

Out[40]:
```
False
```

In [41]:
```
# BEWARE!
x = 5
y = 8

#xx == 6 or xxx == 6 or x > 2
x > 42 and (xx > 1000 or y < 7)
```

Out[41]:
```
False
```

Python does short-circuit evaluation of operators

More on sequences (For example strings and lists)

Lists (and strings) are an ORDERED collection of elements where every element can be accessed through an index.

Operators	Descriptions
x in s	True if an item in *s* is equal to *x*
s + t	Concatenates *s* and *t*
s * n	Adds *s* to itself *n* times
s[i]	*i*th item of *s*, origin 0
s[i:j]	slice of *s* from *i* to *j-1*
s[i:j:k]	slice of *s* from *i* to *j-1* with step *k*

In [42]:
```
l = [2,3,4,5,3,7,5,9]
s = 'some longrandomstring'

#'o' in s
l[0]
s[4:7]
s[0:8:2]
s[-1]
```

```
l[0] = 42
s[0] = 'S'
```

```
---------------------------------------------------------------------
----------
TypeError                                 Traceback (most recent
call last)
Input In [42], in <cell line: 10>()
      8 s[-1]
      9 l[0] = 42
---> 10 s[0] = 'S'

TypeError: 'str' object does not support item assignment
```

Mutable vs Immutable objects

Mutable objects can be altered after creation, while immutable objects can't.

Immutable objects:

- `int`
- `float`
- `bool`
- `str`
- `tuple`

Mutable objects:

- `list`
- `set`
- `dict`

Operations on mutable sequences

Operation	Result
s[i] = x	item *i* of *s* is replaced by *x*
s[i:j] = t	slice of *s* *from* *i* to *j-1* is replaced by the contents of the iterable t
del s[i:j]	removes element *i* to *j-1*
s[i:j:k] = t	specified element replaced by *t*
s.append(x)	appends *x* to the end of the sequence
s[i:j:k]	slice of *s* from *i* *to* *j-1* with step *k*
s[:] or	creates a copy of *s*
s.copy( )	creates a copy of *s*
s.insert(i, x)	inserts *x* *into* *s* at the index *i*
s.pop([i])	retrieves the item *i* from *s* and also removes it
s.remove(x)	retrieves the first item from *s* where s[i] == x
s.reverse( )	reverses the items of *s* in place

In [43]:

```
s = [0,1,2,3,4,5,6,7,8,9]
s.insert(5,10)
s.reverse()
s.append(10)
s
```

Out[43]: [9, 8, 7, 6, 5, 10, 4, 3, 2, 1, 0, 10]

Summary

- The python standard library has many built-in functions regularly used
- Operators are used to carry out computations on different values
- Three types of operators; comparison, logical, and membership
- Order of precedence crucial!
- Mutable object can be changed after creation while immutable objects cannot be changed

→ **Notebook Day_1_Exercise_2 (~30 minutes)**

Loops in Python

In [44]:
```
fruits = ['apple','pear','banana','orange', 'grapes', 'pears']

print(fruits[0])
print(fruits[1])
print(fruits[2])
print(fruits[3])
print(fruits[4])
print(fruits[5])
```

```
apple
pear
banana
orange
grapes
pears
```

Loops in Python

In [44]:
```
fruits = ['apple','pear','banana','orange', 'grapes', 'pears']

print(fruits[0])
print(fruits[1])
print(fruits[2])
print(fruits[3])
print(fruits[4])
print(fruits[5])
```

```
apple
pear
banana
orange
grapes
pears
```

In [45]:
```
fruits = ['apple','pear','banana','orange', 'grapes']

for fruit in fruits:
    print(fruit)
print('DONE!')
```

```
apple
pear
banana
orange
grapes
DONE!
```

Always remember to INDENT your loops!

Different types of loops

Different types of loops

`For` loop

In [46]:
```
fruits = ['apple','pear','banana','orange']
mystring = 'mylongstring'

for fruit in fruits:
    print(fruit)
```

```
apple
pear
banana
orange
```

Different types of loops

`For` loop

In [46]:
```
fruits = ['apple','pear','banana','orange']
mystring = 'mylongstring'

for fruit in fruits:
    print(fruit)
```

```
apple
pear
banana
orange
```

`While` loop

In [47]:
```
fruits = ['apple','pear','banana','orange']

i = 0
while i < len(fruits):
    print(fruits[i])
    i = i + 1

print(i)
```

```
apple
pear
banana
orange
4
```

Different types of loops

`For` loop

Is a control flow statement that performs a fixed operation over a known amount of steps.

`While` loop

Is a control flow statement that allows code to be executed repeatedly based on a given Boolean condition.

Which one to use?

`For` loops better for simple iterations over lists and other iterable objects

`While` loops are more flexible and can iterate an unspecified number of times

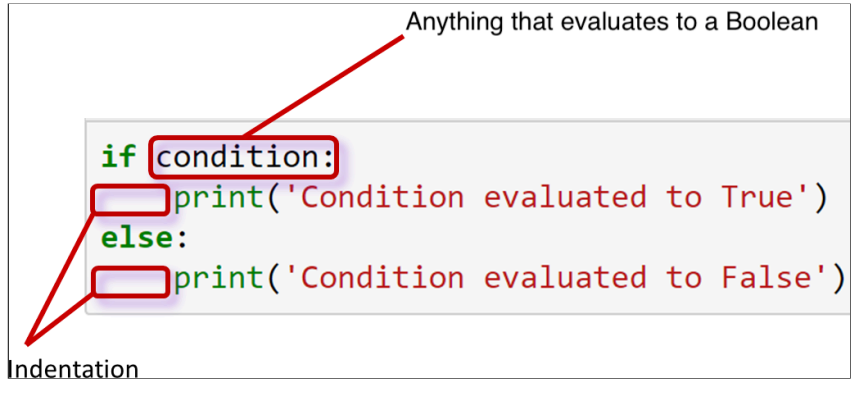
Example of a simple Python script

```python
# A simple loop that adds 2 to a number
i = 0
while i < 10:
    u = i + 2
    print('u is '+str(u))
    i += 1
```

```
u is 2
u is 3
u is 4
u is 5
u is 6
u is 7
u is 8
u is 9
u is 10
u is 11
```

→ **Notebook Day_1_Exercise_3 (~20 minutes)**

Conditional `if/else` statements



In [48]:

```
shopping_list = ['bread', 'egg', 'butter', 'milk']

if len(shopping_list) > 3:
    print('Go shopping!')
```

```
Go shopping!
```

In [48]:
```
shopping_list = ['bread', 'egg', 'butter', 'milk']

if len(shopping_list) > 3:
    print('Go shopping!')
```

```
Go shopping!
```

In [49]:
```
shopping_list = ['bread', 'egg', 'butter', 'milk']
tired         = False

if len(shopping_list) > 5:
    if not tired:
        print('Go shopping!')
    else:
        print('Too tired, I\'ll do it later')
else:
    if not tired:
        print('Better get it over with today anyway')
    else:
        print('Nah! I\'ll do it tomorrow!')
```

```
Better get it over with today anyway
```

In [48]:
```
shopping_list = ['bread', 'egg', 'butter', 'milk']

if len(shopping_list) > 3:
    print('Go shopping!')
```

```
Go shopping!
```

In [49]:
```
shopping_list = ['bread', 'egg', 'butter', 'milk']
tired         = False

if len(shopping_list) > 5:
    if not tired:
        print('Go shopping!')
    else:
        print('Too tired, I\'ll do it later')
else:
    if not tired:
        print('Better get it over with today anyway')
    else:
        print('Nah! I\'ll do it tomorrow!')
```

```
Better get it over with today anyway
```

This is an example of a nested conditional

Putting everything into a Python script

Any longer pieces of code that have been used and will be re-used SHOULD be saved

Two options:

- Save it as a text file and make it executable
- Save it as a notebook file

Things to remember when working with scripts

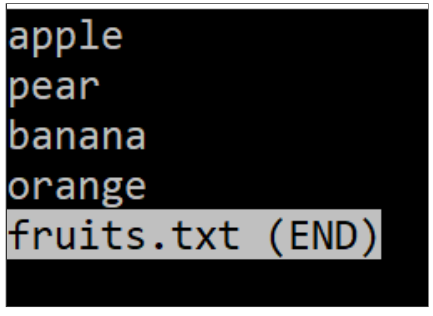
- Put *#!/usr/bin/env python* in the beginning of the file
- Make the file executable to run with `./script.py`
- Otherwise run script with `python script.py`

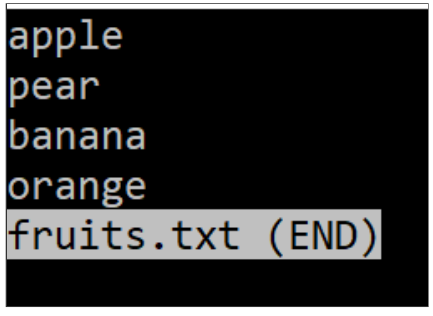
Working on files

In [50]:
```
fruits = ['apple','pear','banana','orange']

for fruit in fruits:
    print(fruit)
```

```
apple
pear
banana
orange
```


apple
pear
banana
orange
fruits.txt (END)



In [51]:
```
fh = open('../files/fruits.txt', 'r', encoding = 'utf-8')

for line in fh:
    print(line)

fh.close()
```

```
apple

pear

banana

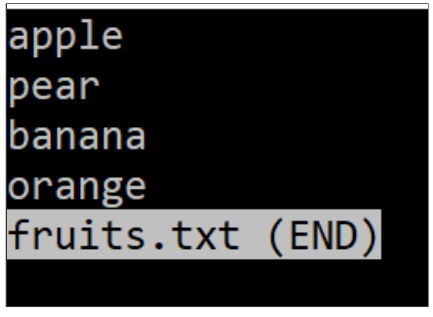
orange
```

Aditional useful methods:

`'string'.strip()`	Removes whitespace
`'string'.split()`	Splits on whitespace into list

In [52]:
```
s   = '  an example string to split with whitespace in end   '
sw  = s.strip()
sw
swl = sw.split()
swl = s.strip().split()
swl
```

Out[52]:
```
['an', 'example', 'string', 'to', 'split', 'with', 'whitespace',
'in', 'end']
```



In [53]:
```
fh = open('../files/fruits.txt', 'r', encoding = 'utf-8')

for line in fh:
    print(line.strip())

fh.close()
```

```
apple
pear
banana
orange
```

Another example

```
ICA     254
Icecream        65
Coop    25.45
ICA     654.21
Pharmacy        39.90
IKEA    2365
ATM     500
SevenEleven     62.60
ICA     278.50
Åhlens  645.20
bank_statement.txt (END)
```

How much money is spent on ICA?

Another example

```
ICA     254
Icecream        65
Coop    25.45
ICA     654.21
Pharmacy        39.90
IKEA    2365
ATM     500
SevenEleven     62.60
ICA     278.50
Åhlens  645.20
bank_statement.txt (END)
```

How much money is spent on ICA?

In [54]:
```
fh    = open("../files/bank_statement.txt", "r", encoding = "utf-8")

total = 0

for line in fh:
    expenses = line.strip().split()  # split line into list
    store    = expenses[0]           # save what store
    price    = float(expenses[1])    # save the price
    if store == 'ICA':               # only count the price if store is
        total = total + price
```

```
fh.close()

print('Total amount spent on ICA is: '+str(total))
```

```
Total amount spent on ICA is: 1186.71
```

Slightly more complex...

```
store    year    month   day     sum
ICA      2018    08      30      254
Icecream         2018    09      05      65
Coop     2018    09      08      25.45
ICA      2018    09      22      654.21
Pharmacy         2018    09      23      39.90
IKEA     2018    09      25      2365
ATM      2018    09      28      500
SevenEleven      2018    09      29      62.60
ICA      2018    09      29      278.50
Åhlens   2018    10      02      645.20
bank_statement_extended.txt (END)
```

How much money is spent on ICA in September?

In [55]:

```
fh    = open("../files/bank_statement_extended.txt", "r", encoding = "ut

total = 0

for line in fh:
    if not line.startswith('store'):
        expenses = line.strip().split()
        store    = expenses[0]
        year     = expenses[1]
        month    = expenses[2]
        day      = expenses[3]
        price    = float(expenses[4])
        if store == 'ICA' and month == '09':   # store has to be ICA and
            total = total + price
fh.close()

out = open("../files/bank_statement_results.txt", "w", encoding = "utf-8
out.write('Total amount spent on ICA in september is: '+str(total))
out.close()
```

In [55]:
```
fh    = open("../files/bank_statement_extended.txt", "r", encoding = "ut

total = 0

for line in fh:
    if not line.startswith('store'):
        expenses = line.strip().split()
        store    = expenses[0]
        year     = expenses[1]
        month    = expenses[2]
        day      = expenses[3]
        price    = float(expenses[4])
        if store == 'ICA' and month == '09':   # store has to be ICA and
            total = total + price
fh.close()

out = open("../files/bank_statement_results.txt", "w", encoding = "utf-8
out.write('Total amount spent on ICA in september is: '+str(total))
out.close()
```

In [56]:
```
for file in os.scandir("../files/"):
    print(time.ctime(os.stat(file).st_mtime), '\t', file.name)
```

```
Tue Oct 11 18:39:02 2022         250.imdb
Thu May 20 17:46:00 2021         bank_statement.txt
Thu May 20 17:46:00 2021         bank_statement_extended.txt
Fri Oct  6 12:35:06 2023         bank_statement_results.txt
Thu May 20 17:46:00 2021         blocket_listings_selected.txt
Thu May 20 17:46:01 2021         cheat_sheet.pdf
Thu May 20 17:46:01 2021         fruits.txt
Thu May 20 17:46:01 2021         fruits_extended.txt
Wed Oct 12 08:43:09 2022         imdb_reformatted.txt
Fri Sep 30 15:40:44 2022         schedule.csv
Thu May 20 17:46:01 2021         somerandomfile.txt
```

In [55]:
```
fh    = open("../files/bank_statement_extended.txt", "r", encoding = "ut

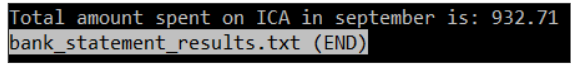
total = 0

for line in fh:
    if not line.startswith('store'):
        expenses = line.strip().split()
        store    = expenses[0]
        year     = expenses[1]
        month    = expenses[2]
        day      = expenses[3]
        price    = float(expenses[4])
        if store == 'ICA' and month == '09':   # store has to be ICA and
            total = total + price
fh.close()

out = open("../files/bank_statement_results.txt", "w", encoding = "utf-8
out.write('Total amount spent on ICA in september is: '+str(total))
out.close()
```

In [56]:
```
for file in os.scandir("../files/"):
    print(time.ctime(os.stat(file).st_mtime), '\t', file.name)
```

```
Tue Oct 11 18:39:02 2022          250.imdb
Thu May 20 17:46:00 2021          bank_statement.txt
Thu May 20 17:46:00 2021          bank_statement_extended.txt
Fri Oct  6 12:35:06 2023          bank_statement_results.txt
Thu May 20 17:46:00 2021          blocket_listings_selected.txt
Thu May 20 17:46:01 2021          cheat_sheet.pdf
Thu May 20 17:46:01 2021          fruits.txt
Thu May 20 17:46:01 2021          fruits_extended.txt
Wed Oct 12 08:43:09 2022          imdb_reformatted.txt
Fri Sep 30 15:40:44 2022          schedule.csv
Thu May 20 17:46:01 2021          somerandomfile.txt
```



Summary

- Python has two types of loops, `For` loops and `While` loops
- Loops can be used on any iterable types and objects
- `If/Else` statement are used when deciding actions depending on a condition that evaluates to a boolean
- Several `If/Else` statements can be nested
- Save code as notebook or text file to be run using python
- The function `open()` can be used to read in text files
- A text file is iterable, meaning it is possible to loop over the lines

→ **Notebook Day_1_Exercise_4**